

Analisi degli attacchi

Protocollo di voto elettronico per referendum di Ateneo — Gruppo 7
Mappatura fra gli attacchi delle slide del corso e le garanzie effettive del prototipo (WP4)

Questo documento mette in corrispondenza ogni classe di attacco presentata nelle slide del corso (01_Introduction, 04_Integrity, 05_Public_Key, 06_Key_Distribution, 07_Blockchain, 08_User_Authentication) con il comportamento effettivo del protocollo progettato in WP2, analizzato in WP3 e implementato in WP4. Per ciascun attacco si indica se il sistema lo blocca, lo blocca solo in parte, non lo blocca affatto, oppure se l'attacco non è applicabile a questo modello, sempre motivando la risposta con il meccanismo concreto (e, dove utile, con il file del prototipo che lo realizza).

Criterio di onestà adottato. Le voci marcate NON MITIGATO non sono dimenticanze: sono limiti che WP3 dichiara esplicitamente (rischi T.1–T.6) o conseguenze dirette di assunzioni di fiducia dichiarate (F.1–F.9). La traccia del project work chiede espressamente "un'analisi critica onesta dei limiti del sistema" e valuta "la consapevolezza critica dei compromessi, non la completezza della soluzione": dichiarare cosa il sistema non fa vale quanto dimostrare cosa fa.

MITIGATO	Il protocollo lo blocca per costruzione, con un meccanismo crittografico o architetturale verificabile.
PARZIALE	Blocca lo scenario principale ma lascia un residuo dichiarato, oppure dipende da un'assunzione operativa.
NON MITIGATO	Fuori dal perimetro delle garanzie: limite dichiarato in WP3, non un'omissione.
NON APPLICABILE	L'attacco non ha senso in questo modello (meccanismo non presente nel sistema).

1. Attacchi generali (slide 01 – Introduction)

La slide 01 elenca le classi in cui "most attacks fall": identity theft, phishing, denial-of-service, ransomware, cyber-extortion, cyber-warfare, hacktivism, crypto-jacking, supply-chain attack, più la nozione di zero-day.

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Identity theft	PARZIALE	L'autenticazione challenge-response (§2.2.5) non trasmette mai la password in chiaro e il backoff esponenziale dopo $k=3$ tentativi limita il brute force online. Residuo: non c'è un secondo fattore, quindi chi ottiene le credenziali vota al posto dell'elettore; inoltre l'IdP memorizza $h_pwd = H(password \parallel salt)$, una credenziale <i>plaintext-equivalent</i> (T.5): chi esfiltra il database si autentica senza conoscere la password.
Phishing	NON MITIGATO	Nessun meccanismo crittografico previene un sito o un client clone: è un attacco all'utente, non al protocollo. Aggancio disponibile: il manifest firmato (§2.4.3) pubblica gli endpoint ufficiali e l'hash del client di voto legittimo, quindi un elettore che verifichi il manifest ha un riferimento non falsificabile — ma la verifica dipende da lui.
Denial-of-service	NON MITIGATO	Dichiarato in A.1 (§3.2): IdP e Ballot Server restano single point of failure, la difesa spetta all'infrastruttura. Il protocollo garantisce però <i>robustezza</i> : un rinvio dopo timeout non produce doppio voto (tabella Used). Lato scrutinio, la soglia (3,5) tollera 2 commissari indisponibili; con 3 o più indisponibili lo scrutinio fallisce in modo <i>fail-safe</i> (nessun voto viene esposto in chiaro).
Ransomware	NON APPLICABILE	Attacco all'infrastruttura, non al protocollo. Va però segnalata una conseguenza specifica: le share sono l'unico backup della chiave di decifratura, quindi un ransomware che colpisca i dispositivi di 3 commissari rende lo scrutinio impossibile (§3.2, "perdita irreversibile delle share").
Cyber-extortion (coercizione)	PARZIALE	La ricevuta contiene ID_seq, head_i e $H(C \parallel ID_seq)$ ma non il voto, e la codifica canonica a lunghezza fissa impedisce il signaling: il coercitore <i>post hoc</i> non ottiene alcuna prova del contenuto (C.3). Non copre: il compratore con elettore consenziente (T.3), la coercizione durante la sessione, e l'assenza di revoca (R.1) che preclude il ri-voto in privato.
Cyber-warfare / hacktivism	PARZIALE	Avversari a risorse elevate: il protocollo regge finché reggono le assunzioni F.1–F.9. Un avversario capace di compromettere l'IdP (F.4) o $t=3$ commissari (F.2) rompe rispettivamente C.2 e C.1. La soglia (3,5) e la separazione IdP/BS alzano il costo dell'attacco, non lo rendono impossibile.
Crypto-jacking	NON APPLICABILE	Nessun proof-of-work e nessuna risorsa computazionale da dirottare: il Bulletin Board è un registro append-only, non una blockchain con mining.

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Supply-chain attack	PARZIALE	§2.3 dichiara che la distribuzione del client come software open source, firmato e con hash verificabile, mitiga la consegna di un binario malevolo. Nel prototipo l'hash del client ufficiale è un campo del manifest firmato da tutti i commissari e dalla CA, quindi verificabile. Non mitiga un trojan già residente sull'endpoint (T.2): lì il binario in esecuzione è quello giusto ed è l'ambiente a essere ostile.
Zero-day	NON MITIGATO	Per definizione non mitigabile. Nota di perimetro: il prototipo si appoggia alla libreria <i>cryptography</i> per RSA/X.509/OAEP, quindi eredita le vulnerabilità non note di quella libreria e di OpenSSL sottostante.

2. Integrità, MAC e attacchi al padding (slide 04 – Integrity)

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Forgery di una firma (Mac-forge)	MITIGATO	Tutte le firme del protocollo usano RSA hash-and-sign (PKCS#1 v1.5 + SHA-256): <code>sigma_token</code> , <code>sigma_voter</code> , ricevuta, teste periodiche, tally bundle. Un avversario che non possieda la chiave privata non produce un tag valido. (<i>src/common/signing.py</i>)
Replay attack	MITIGATO	È l'attacco su cui il sistema investe di più, con due barriere indipendenti. In autenticazione, la nonce fresca <code>n1</code> a 128 bit rende inutilizzabile una response catturata (§2.5.1). Nel voto, la tabella <i>Used</i> del Ballot Server rifiuta un <code>token_id</code> già consumato (l.2). La slide 04 osserva che "a MAC by itself cannot protect against such attacks since verification is stateless": il protocollo aggiunge esattamente lo stato che manca. Dimostrato dal vivo in <i>demo_election.py</i> .
Length extension attack	NON APPLICABILE	SHA-256 è vulnerabile all'estensione di lunghezza, ma il protocollo non usa mai la costruzione insicura $H(\text{segreto} \parallel \text{messaggio})$ come MAC. Nell'unico punto in cui un segreto entra in un hash con un valore noto, la response è $H(n1 \parallel h_pwd)$: il segreto è in cod a, posizione in cui l'estensione non aiuta l'attaccante. La forma $H(\text{password} \parallel \text{salt})$ ha invece il segreto in testa, ma nessun messaggio del protocollo ha la struttura che servirebbe per sfruttarla — e chi conoscesse <code>h_pwd</code> avrebbe già vinto per altra via (T.5).
Padding oracle attack	MITIGATO	Doppia difesa. Architetturale : nessuna entità raggiungibile da un attaccante decifra su richiesta — il Ballot Server non possiede <code>sk_BS</code> , e la Commissione decifra una sola volta, in blocco, a urne chiuse, su dispositivo isolato, senza restituire esiti a nessuno: l'oracolo non esiste. Implementativa : la decifratura OAEP scritta a mano (RFC 8017) combina i quattro controlli di validità con <code>&</code> non a corto circuito e confronta l'hash del label con <code>hmac.compare_digest</code> , per non trapezare <i>quale</i> controllo sia fallito (attacco di Manger). Residuo dichiarato : la scansione del separatore resta a tempo variabile. (<i>src/common/rsa_raw.py</i>)
Chosen-ciphertext attack (CCA)	MITIGATO	RSA-OAEP è IND-CCA2 nel Random Oracle Model. Coerentemente, il modello non espone alcun oracolo di decifratura: il gestore del Ballot Server vede solo ciphertext e non ha la chiave privata (§3.3.1).
Malleabilità	MITIGATO	OAEP non è malleabile, e in più <code>sigma_voter</code> firma $H(C \parallel \text{token_id} \parallel \text{head_ref})$: alterare <code>C</code> in transito invalida la firma dell'elettore e la scheda viene scartata al passo 4 di §2.7 (l.3). Lo scenario è riprodotto nella demo con un intercettatore che sostituisce il ciphertext.
Side-channel sui messaggi d'errore	MITIGATO	I controlli del Ballot Server si fermano al primo fallimento con errori tipizzati, ma la distinzione non aiuta l'attaccante: nessuno dei messaggi rivela informazione sul <i>contenuto</i> del voto, che resta cifrato. Sul lato password il confronto della response usa <code>hmac.compare_digest</code> (tempo costante).

3. Crittografia a chiave pubblica (slide 05 – Public Key)

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
RSA deterministico su spazio dei messaggi piccolo	MITIGATO	È l'attacco più rilevante per un referendum ed è il motivo per cui §2.2.1 impone OAEP. I messaggi possibili sono due: con RSA "testuale" chiunque cifrerebbe YES e NO con la chiave pubblica nota e confronterebbe i risultati con i ciphertext del Bulletin Board, leggendo ogni voto. Il seme <code>r_enc</code> fresco a ogni cifratura rende i ciphertext indistinguibili anche su uno spazio di due soli messaggi (C.1). La slide 05 descrive esattamente questa fuga di informazione.
Fattorizzazione di N / brute force	MITIGATO	Parametri conformi allo stato dell'arte: modulo RSA a 2048 bit, $e = 65537$, chiavi generate da OpenSSL tramite <i>cryptography</i> . Nessuna scorciatoia matematica nota alla portata di un avversario realistico.
No-message forgery ($m' = \text{sigma}^e \bmod N$)	MITIGATO	La firma non è su un messaggio arbitrario ma su un digest SHA-256 con padding PKCS#1 v1.5. Per sfruttare la forgery l'attaccante dovrebbe trovare un messaggio il cui SHA-256 coincida con sigma^e : una preimmagine, computazionalmente infattibile. È precisamente la ragione per cui la slide 05 prescrive hash-and-sign.
Existential forgery moltiplicativa ($\text{sigma1} \cdot \text{sigma2}$)	MITIGATO	Stessa difesa: la malleabilità moltiplicativa di RSA testuale non si trasferisce attraverso SHA-256 e il padding strutturato, perché il prodotto di due firme non corrisponde all'hash di alcun messaggio scelto dall'attaccante.

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Signature stripping e sostituzione	MITIGATO	La slide 05 descrive l'avversario che rimuove la firma di S e la sostituisce con la propria. Il protocollo lo blocca nel punto critico legando crittograficamente token e chiave: sigma_token firma $H(\text{token_id} \text{pk_voter})$, quindi chi intercetta (token_id, sigma_token) non può usarlo con una propria coppia di chiavi, perché la firma non verificherebbe (§2.2.2).
Ciphertext replaying	MITIGATO	Il reinvio di un ciphertext già registrato viene fermato dalla tabella <i>Used</i> prima ancora di arrivare alla verifica di integrità (passo 3 di §2.7, che precede il passo 4).

4. Distribuzione delle chiavi e PKI (slide 06 – Key Distribution)

È la sezione su cui il manifest di elezione (§2.4.3) incide di più: senza un oggetto firmato che fissi tutte le chiavi pubbliche, ogni parametro arriverebbe a client e osservatori "sulla parola", cioè dallo stesso canale che si vorrebbe poter verificare.

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Man-in-the-middle / sostituzione di chiave pubblica	MITIGATO	La slide 06 avverte che chiunque può falsificare l'annuncio di una chiave pubblica e che senza autenticazione lo scambio è vulnerabile al MITM. Qui le chiavi non sono annunciate ma certificate: la CA di Ateneo emette certificati X.509 con Key Usage differenziato per ruolo, e tutte le chiavi pubbliche dell'elezione (pk_BS, pk_IdP, pk_BS-srv, certificati di trasporto) sono elencate nel manifest firmato da 5 commissari e dalla CA. Sostituire una chiave richiederebbe di falsificare quelle firme.
Fiducia "sulla parola" in una chiave	MITIGATO	Il Ballot Server non accetta una pk_IdP passata come parametro: parte dal certificato, ne verifica autenticità, validità temporale e stato di revoca contro la CA, controlla che il Key Usage abbia digitalSignature e nonRepudiation (il solo digitalSignature è acceso anche sul certificato TLS) e solo allora estrae la chiave. (<i>verify_token_as_ballot_server</i>)
Compromissione del KDC / "complete break"	PARZIALE	§3.3.2 scarta esplicitamente il pattern a bearer token con introspezione in tempo reale, proprio per non ricreare il legame che si vuole evitare. L'analogo strutturale è l'IdP: la sua compromissione rompe C.2, I.1 e I.2 (F.4), ed è un'assunzione dichiarata, non mitigata. Sul lato chiave di decifratura la risposta di design c'è: la soglia (3,5) elimina il singolo detentore, quindi non esiste un'entità la cui compromissione riveli i voti.
Insider attack	PARZIALE	La separazione IdP/BS realizza il least privilege: l'IdP conosce l'identità ma non il voto, il BS il voto cifrato ma non l'identità, quindi nessun singolo insider correla i due. Resta il caso dell'insider all'IdP con accesso alla tabella Issued prima della sua distruzione (F.4, T.5).
Blocco della revoca da parte di un attaccante attivo	PARZIALE	Il ciclo di revoca e CRL è implementato e dimostrato (un token resta valido crittograficamente ma viene rifiutato dopo la revoca della chiave elettorale dell'IdP). Il limite è dichiarato nel codice: una CRL non disponibile significa "non so nulla sulla revoca", non "nessuna revoca", quindi un attaccante che blocchi la CRL degrada la verifica. OSCP è escluso di proposito (sarebbe un ulteriore single point of failure).
Coppersmith / ROCA	NON APPLICABILE	Riguarda chiavi prodotte da generatori difettosi con esponenti particolari. Le chiavi del prototipo sono generate da OpenSSL con $e = 65537$ e moduli a 2048 bit.

5. Registro pubblico (slide 07 – Blockchain)

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Manomissione retroattiva del registro	PARZIALE	Il Bulletin Board è una catena di hash: $head_i = H(head_{i-1} ID_{seq} C pk_{voter} token_{id})$. Modificare una entry passata cambia la sua testa e tutte le successive, quindi ogni modifica <i>parziale</i> è rilevabile ricalcolando la catena (chiunque può farlo con le sole entry pubbliche e head_0 preso dal manifest). Limite dichiarato (I.3, F.8): una riscrittura <i>integrale e coerente</i> da parte del Ballot Server produrrebbe una catena internamente valida. La contromisura non è matematica ma cooperativa: le teste periodiche firmate archiviate da osservatori indipendenti (§2.7.1), che rendono la manipolazione pubblicamente dimostrabile.
51% attack / double spending	NON APPLICABILE	Non c'è consenso distribuito da attaccare: il registro è append-only e scritto da una sola entità, scelta dichiarata come single point of failure in §3.2. La domanda "perché non una blockchain?" ha una risposta precisa: per un referendum di Ateneo il costo di un consenso distribuito non è giustificato, e il ruolo che li svolgono i miner qui è svolto dagli osservatori che archiviano le teste firmate.
Fork del registro	NON APPLICABILE	Non essendoci consenso distribuito non esistono fork nel senso della slide 07. L'equivalente funzionale è la divergenza fra una testa firmata archiviata da un terzo e il registro pubblicato: è esattamente la prova che rende la manipolazione non ripudiabile.

6. Autenticazione dell'utente (slide 08 – User Authentication)

Attacco	Stato	Come lo gestisce (o perché no) il protocollo
Brute force online	MITIGATO	Contatore di tentativi falliti per matricola e backoff esponenziale $2 \cdot 2^{(\text{tentativi}-3)}$ secondi dopo $k = 3$ fallimenti consecutivi (§2.5.1). Dimostrato in <code>demo_idp.py</code> .
Precomputation / rainbow tables	MITIGATO	Salt casuale di 32 byte, univoco per credenziale: gli hash di password identiche presso utenti diversi restano indipendenti e nessuna tabella precalcolata è riutilizzabile. È esattamente la difesa che la slide 08 attribuisce al salt.
Dictionary / brute force offline su database esfiltrato	NON MITIGATO	La slide 08 distingue due attacchi, "precomputation" e "dictionary/brute-forcing": il salt copre il primo, non il secondo. Il protocollo memorizza $h_pwd = H(\text{password} \text{salt})$ con un solo passaggio di SHA-256, quindi un attaccante che esfiltri il database prova miliardi di candidati al secondo. È il limite T.5 , dichiarato: una KDF iterata (PBKDF2, bcrypt) ridurrebbe il rischio, non adottata per restare entro le primitive di hashing salato trattate a corso. Aggravante: h_pwd è <i>plaintext-equivalent</i> , cioè basta a superare l'autenticazione anche senza invertire l'hash.
Password spraying	NON MITIGATO	Segnalazione emersa dall'analisi del codice, non presente in WP3: il contatore di tentativi e il backoff sono per matricola . Un attaccante che provi <i>una sola</i> password comune su molte matricole diverse non innesca mai la soglia $k = 3$ su nessuna di esse. La mitigazione naturale sarebbe un limite aggregato per finestra temporale o per indirizzo di origine, che il prototipo non implementa.
Credential stuffing / credenziali trapelate altrove	NON MITIGATO	Nessun secondo fattore: chi possiede le credenziali valide è indistinguibile dall'elettore legittimo. Coerente con la scelta dichiarata in §2.2.5 di non richiedere authenticator hardware dedicati.
Replay della risposta di autenticazione	MITIGATO	La response è $H(n1 h_pwd)$ con $n1$ fresca a ogni sessione: nessuna delle quattro varianti di replay elencate nella slide 08 (simple replay, repetition that can be logged, backward replay, suppress-replay) è riutilizzabile, perché la nonce non si ripete.
Impersonazione via handshake intercettato	MITIGATO	La password non transita mai in chiaro; il canale verso IdP e BS è cifrato e autenticato (F.9); il token emesso è legato crittograficamente a pk_voter , quindi non è trasferibile a un'altra coppia di chiavi.
Timing attack sul confronto del segreto	MITIGATO	Il confronto della response usa <code>hmac.compare_digest</code> , a tempo costante — la stessa lezione del caso Xbox 360 citata a corso. Non era richiesto esplicitamente dal documento di progetto.
Phishing delle credenziali	NON MITIGATO	Vedi sezione 1: fuori dal perimetro del protocollo.

7. I limiti dichiarati: cosa il sistema non protegge, e perché

Le voci NON MITIGATO delle tabelle precedenti si riconducono a pochi limiti strutturali, già isolati in WP3 come rischi trasversali. Vale la pena saperli enunciare in una frase ciascuno, perché è esattamente ciò che la traccia chiede di dimostrare.

T.2 — Client dell'elettore compromesso

È l'unico avversario che da solo, senza alcuna collusione, rompe più proprietà indipendenti: esfiltra il voto in chiaro e r_enc (C.1), fornisce una prova trasferibile a un coercitore (C.3) e può firmare con sk_voter una preferenza diversa da quella scelta. Quest'ultimo caso non viola il non-ripudio in senso stretto (la firma resta valida) ma spezza la corrispondenza fra intenzione ed espressione, e l'elettore non è in grado di accorgersene, perché la verifica individuale si ferma al ciphertext. Nessuna difesa lato server è possibile: la firma del binario copre la catena di distribuzione, non un ambiente di esecuzione già ostile.

T.3 — Vendita volontaria del voto

La distruzione di r_enc avviene sul dispositivo dell'elettore, sotto il suo pieno controllo, e il protocollo non ha modo di verificarla. Chi voglia vendere il voto semplicemente non la esegue e consegna al compratore (C, ID_seq , r_enc , v): ricifrando si ottiene una prova crittografica completa e trasferibile. Di conseguenza C.3 è enunciata in forma restrittiva — resiste al coercitore *post hoc*, non al compratore con elettore consenziente.

T.4 — Correttezza della decifratura non verificabile

Una commissione con t membri corrotti che dichiara un totale falso non viene rilevata da nessun controllo pubblico: il conteggio è interno al bundle firmato dalla commissione stessa. Il limite non è eliminabile localmente, perché renderlo verificabile richiederebbe di pubblicare r_{enc} per ogni scheda (distruggendo C.3 in modo generalizzato) oppure prove a conoscenza zero / mixnet verificabili, strumenti fuori dal perimetro del corso. Restano verificabili autenticità e completezza: che ogni scheda contata sia autorizzata, e che il numero di schede decifrate coincida con quelle sul registro.

T.5 — Primitiva di autenticazione debole in caso di data breach

$h_{pwd} = H(\text{password} || \text{salt})$ è plaintext-equivalent e resiste male a un attacco offline. Il rischio è trasversale: un attaccante che ottenga anche un accesso temporaneo alla tabella Issued prima della cancellazione completerebbe il legame matricola-chiave effimera, minacciando C.2.

Password spraying (non in WP3)

Il rate limiting per matricola non ferma un attacco che distribuisce i tentativi su molte matricole. È l'unica osservazione di questa analisi che non era già documentata: vale la pena aggiungerla a WP3 come rischio residuo, o implementare un limite aggregato, prima della consegna.

8. Domande probabili in sede di discussione

Risposte già pronte alle domande che questa analisi rende naturali.

« Perché RSA-OAEP e non RSA "normale"? »

Perché il referendum ha due soli messaggi possibili. Con RSA deterministico chiunque cifrerebbe YES e NO con la chiave pubblica, che è pubblica per definizione, e confronterebbe con i ciphertext del Bulletin Board: la segretezza del voto cadrebbe immediatamente. Il seme fresco di OAEP è ciò che rende due cifrature dello stesso voto indistinguibili.

« Il vostro sistema è vulnerabile al padding oracle? »

No, per due ragioni indipendenti. La prima è architetturale: non esiste un'entità raggiungibile che decifri su richiesta: il Ballot Server non ha la chiave, e la Commissione decifra una sola volta a urne chiuse su un dispositivo isolato. La seconda è implementativa: nella decifrazione OAEP scritta a mano i controlli di validità non sono a corto circuito e il confronto dell'hash usa `compare_digest`, per non rivelare quale controllo sia fallito. Resta un residuo dichiarato sulla scansione del separatore.

« Cosa succede se un commissario è disonesto? »

Nulla: serve la collusione di almeno 3 su 5 per ricostruire la chiave. La soglia tollera due commissari disonesti o indisponibili. Se invece 3 o più sono indisponibili lo scrutinio diventa impossibile, ma è un fallimento fail-safe: i voti non vengono esposti. Preferiamo questo a un fail-open.

« Come fa un elettore a sapere che il suo voto è stato contato? »

Può verificare che la sua scheda cifrata sia registrata: ritrova la entry tramite `ID_seq`, confronta `H(C || ID_seq)` con la ricevuta firmata dal Ballot Server e ricalcola la catena di hash. Quello che **non** può fare è seguire la propria preferenza fino al totale, perché il tally è una lista mescolata senza riferimenti. È il prezzo diretto della non-correlabilità: se potesse seguirla, potrebbe anche dimostrarla a un coercitore.

« Perché non avete usato una blockchain? »

Perché il problema che una blockchain risolve, il consenso distribuito fra entità che non si fidano, qui non si pone: il registro ha un solo scrittore, dichiarato. La proprietà che serviva davvero, l'append-only verificabile, è ottenuta con una catena di hash più l'archiviazione indipendente delle teste firmate. Va detto però che questa è una garanzia cooperativa, non matematica: se nessuno archivia, una riscrittura integrale del registro non sarebbe rilevabile.

« Qual è l'attacco che vi preoccupa di più? »

Il client compromesso (T.2). È l'unico avversario che da solo rompe tre proprietà insieme, ed è anche quello contro cui il protocollo non può fare nulla per costruzione: può leggere la preferenza prima che venga cifrata, conservare la randomness e firmare un voto diverso da quello scelto, senza che l'elettore possa accorgersene.

Riferimenti: le sigle C.x, I.x, V.x, A.1, R.1, E.1 indicano le proprietà di WP1; T.1–T.6 i rischi trasversali di §3.7; F.1–F.9 le assunzioni di fiducia consolidate in §3.8. I riferimenti §x.y rimandano al documento di progetto (WP1–WP3); i nomi di file al prototipo WP4.